

**Question 1: Tokenization (5 marks). The word "UNHAPPINESS" is uncommon enough that a**

**sub-word tokenizer would likely split it into smaller pieces rather than keep it whole.**

**(a) Split "UNHAPPINESS" into 3 reasonable sub-word pieces (there's no single "correct" answer**

**— justify your split).**

**(b) In 1-2 sentences, explain WHY sub-word tokenization (rather than whole-word tokenization)**

**helps a model handle a word like this one, even if the model has never seen the whole word**

**before in training.**

**(a) Split "UNHAPPINESS" into 3 reasonable sub-word pieces**

A reasonable split is:

**UN + HAPPY + NESS**

**Justification:**

- **UN** → Prefix meaning "not" or "opposite of."
- **HAPPY** → Root word.
- **NESS** → Suffix that changes an adjective into a noun, meaning "the state or quality of."

**(b) Why does sub-word tokenization help? (1–2 sentences)**

Sub-word tokenization helps because the model can understand the meaning of a new word by combining the meanings of its smaller parts (such as prefixes, roots, and suffixes). Even if "UNHAPPINESS" was never seen during training, the model can recognize "UN", "HAPPY", and "NESS" and infer the word's meaning.

**Question 2: Embeddings (5 marks). Consider the word pairs “cat” and “kitten” and “cat” and “car”.**

**(a) Which pair would you expect to have HIGHER cosine similarity in embedding space, and why?**

**(b) In your own words, what does an embedding actually represent, and how is it different from a one-hot vector?**

**(a) Which pair would have higher cosine similarity, and why?**

The pair "cat" and "kitten" would have higher cosine similarity because they are semantically related. A kitten is a young cat, so their meanings are very similar, unlike "cat" and "car", which are unrelated.

**(b) What does an embedding represent, and how is it different from a one-hot vector?**

An embedding is a dense numerical vector that represents the meaning and relationships of a word based on its context in language. Similar words have similar embeddings.

A one-hot vector only identifies a word by its position in the vocabulary and does not contain any information about its meaning or similarity to other words.

**Question 3: Self-Attention Worked Example (5 marks). Read the sentence: "The trophy didn't fit**

**in the suitcase because it was too big."**

**(a) The word "it" should attend most strongly to which other word in the sentence? Circle one:**

**"trophy" / "suitcase"**

**(b) Explain in 2-3 sentences WHY self-attention (not a simple word-order rule) is needed to**

**correctly resolve what "it" refers to here.**

**(a) The word "it" should attend most strongly to:**

**A. Trophy**

**(b) Why is self-attention needed?**

Self-attention helps the model understand the relationship between words based on their meaning and context, not just their position in the sentence. In this sentence, "it" refers to the trophy because the trophy is too big to fit in the suitcase. A simple word-order rule might incorrectly choose "suitcase", but self-attention uses the full sentence context to identify the correct reference.

**Question 4: Query, Key, Value (5 marks).** In your own words, briefly explain what the Query,

**Key, and Value vectors each represent in self-attention, and how they combine to produce an attention score.**

- **Query (Q):** Represents what the current word is looking for or searching for in other words.
- **Key (K):** Represents the information each word provides so other words can compare with it.
- **Value (V):** Contains the actual information or representation that is passed to the output if the word receives attention.

**How they produce an attention score:**

The Query of the current word is compared with the Keys of all words to calculate attention scores. Higher scores mean the words are more relevant, so their Value vectors contribute more to the final representation of the current word.